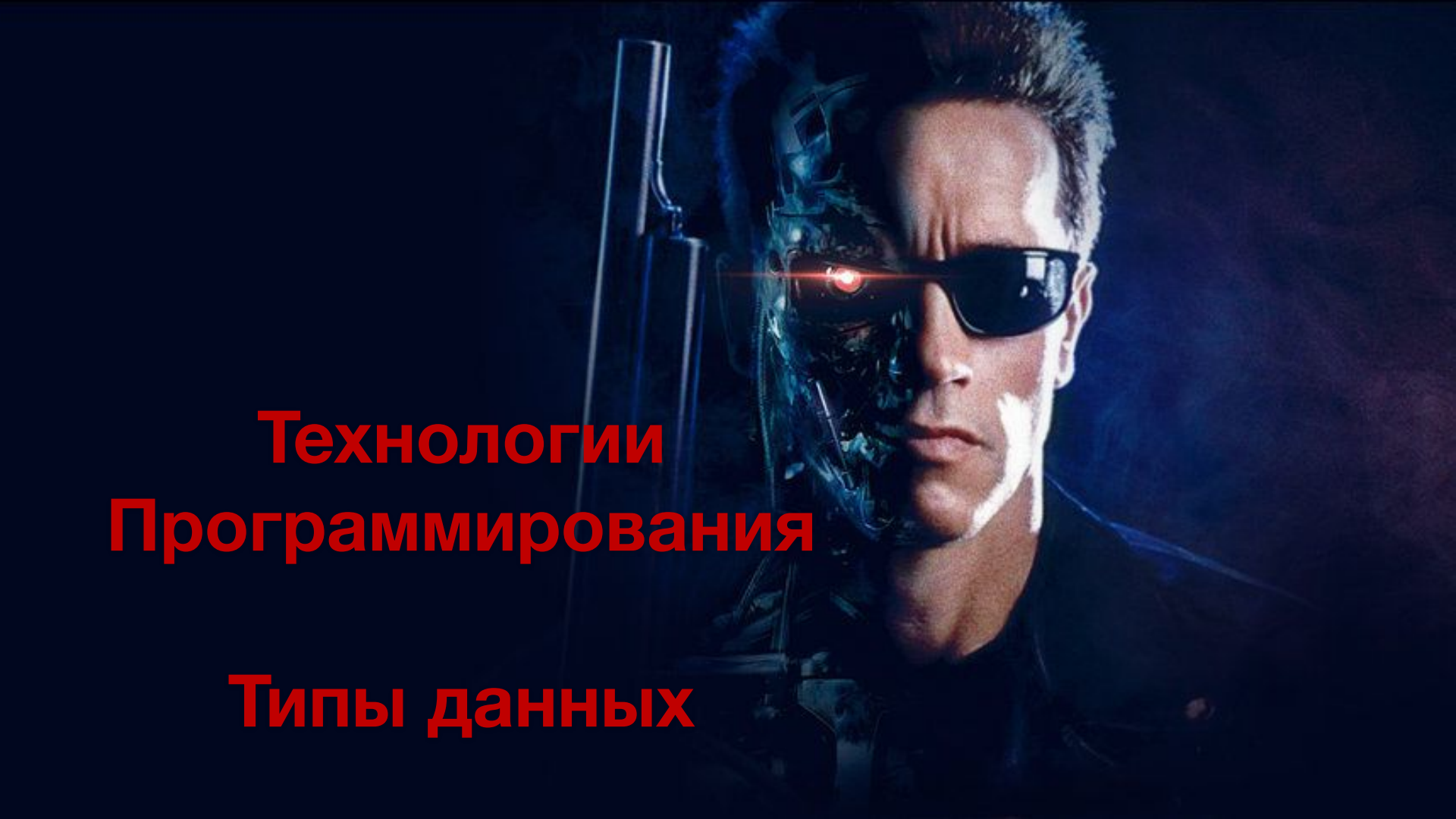




**Технологии
Программирования**

Типы данных

___DISCLAIMER___

Программирование на современном языке C++ напоминает усердную стрельбу себе по ногам.

SCAN LEVEL

2341

STANDARDS:

CXX11
CXX14
CXX17
CXX20
CXX23

VISUAL:
MODEL 956

ANALYSIS: MATCH:

389 VEHI	55378
690 SIZE	38022
600 TSPD	23022
287 HPWR	12048
105 CODE	20673
798 RNGE	29687

Существующие подходы не помогают разработчику, а лишь определяют тот арсенал, из которого он будет с завидным упорством отстреливать свои собственные ноги.

Будем ориентироваться на стандарт Cxx17

Fate

Структура программы:.....similar

- h - header
- cpp - source file
- hpp - header with impl
- ...

-
-
-
-

Синтаксис:.....similar

- Шаблоны
- Лямбды

-
-

Ключевые слова:.....similar

- Список расширен

-

Типы данных:.....similar

- **Или нет.....**

-



Вопрос!

```
int main(int argc, char** argv)
{
    auto result = 5r + 6r;
    printf("result of \"5r + 6r\" is %s\n", (const char*)result);

    return 0;
}
```

Что будет на экране?

```
PS D:\Development\Education\build\bin\Debug> . "D:/Development/Education/build/bin/Debug/types.exe"
result of "5r + 6r" is 56
```

Main

```
int main(int argc, char** argv)
```

Основная функция программы (Entry point).

Без изменений...



Можно поменять в настройках проекта

What are you?

Type Deduction или Выведение Типа

Определение типа данных, типа текущей операции или функции исходя из контекста ее вызова.



Иногда крайне сложно понять что за тип перед вами. Нужно внимательно читать код.

auto

```
auto result = 5r + 6r;
```

- auto - автоматическое определение типа исходя из контекста
- Может быть return value для функции

```
auto DoSomething(int a)
{
    if (a > 0)
    {
        return a; // int DoSomething(int a);
    }

    return "invalid"; // ошибка. мы уже определили return value как int... или нет...
}
```



Применять аккуратно, сильно обфусцирует код

operator

```
result = 5r + 6r;
```

= оператор присваивания

+ оператор сложения

Оператор - это специфическая функция с определенным синтаксисом

- Унарный оператор =, -, new...
- Бинарный оператор +, -, ==, !=...
- Оператор приведения типа, int, *
- Оператор литерала



operator in use



Унарный оператор:

```
real& operator= (uint32_t num)
{
    field = num;
}
```

Бинарный оператор:

```
string operator+ (real a, real b)
{
    return string(std::to_string(a.field) + std::to_string(b.field));
};
```

Оператор приведения:

```
struct real
{
    operator int () { return (int)field; }
    uint32_t field;
};
```

Literal

```
int i = 5r;
```

String literal - суффикс определяющий тип написанного числа

- U для unsigned int
- f для float
- . для double



Literal это **operator** “”

специфическая функция с определенным синтаксисом

```
real operator ""r(unsigned long long a) { return real(a);};
```

Struct

```
struct real
{
    uint32_t field;           // поле структуры
    auto DoSomething()        { return field; }      // функция структуры
    operator int ()           { return (int)field; }  // оператор приведения
};
```

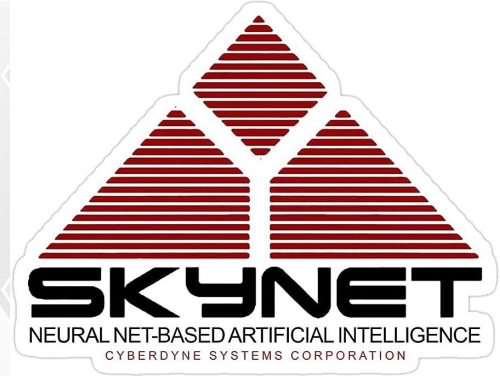
Структура обладает большим количеством новых функций:

- Поля
- Функции с доступом к полям структуры *
- Операторы
- Модификаторы доступа *
- Структуры

```
real number = {2U};          // инициализация
number.DoSomething();         // доступ к функции
number.field = 3;             // доступ к полю
int i = number;               // вызов оператора приведения
```



STL - Standard Template Library



Библиотека типов данных, контейнеров

- `std::string` - работа со строками
- `std::vector` - динамический массив

Набор алгоритмов для работы с данными

- `std::to_string` - конверсия любого типа в строку
- `std::find` - поиск по контейнеру
- `std::sort` - сортировка

И много чего еще...

- `std::thread`
- `std::future`

```
string operator+ (real a, real b)
{
    std::string a_str = std::to_string(a.field);
    std::string b_str = std::to_string(b.field);
    return string(a_str + b_str);
};
```

THANK YOU!

